Why S0 of run 39307409409 failed

Why S0 of run 39307409409 failed (diagnosis, no timing)

Finding: the native side was wrong, and only in the v1 ctypes binding in `arm.sage`. The binding passed the reciprocal of the field modulus to the native context. Sage's solve (`tau.sage`, the baseline) and the Rust solve in `lib.rs` are both correct. The archived n = 19 baseline row is not affected.

The seven systems

S0 of run 39307409409 drew, for t = 0..299, `xR = F.from_integer(rng.randrange(1, 2^19))` and `j = t mod 19`, with `rng = random.Random('volcano-n19-native-tau6-20261019:selftest')` (v1 `arm.sage selftest_row`). `s0_rederive.py` re-derives the 300 systems from that seed and solves each with the archived runner library (`run_github_39307409409/lib/libtau6native.so`, sha256 `b4c6d13d...58e1`) and with a pure-Python S3 evaluator over all 4,096 (u, w) that shares no code with Sage or `lib.rs` (`s0_v1_rederived.json`).

t	xR	j	S3 = 0 over F_2[z]/(m) (Sage's field)	v1 native output
36	14743	17	(u, w) = (2, 54)	none
60	400912	3	(16, 20)	none
118	465769	4	none	(41, 18)
161	59034	9	none	(55, 50)
189	453846	18	none	(56, 23)
195	480552	5	none	(17, 25)
198	486489	8	none	(13, 58)

u and w are the bit vectors of x1 = Σ u_i z^i and of w in x2 = $\sigma^j(\Sigma$ w_l z^l).

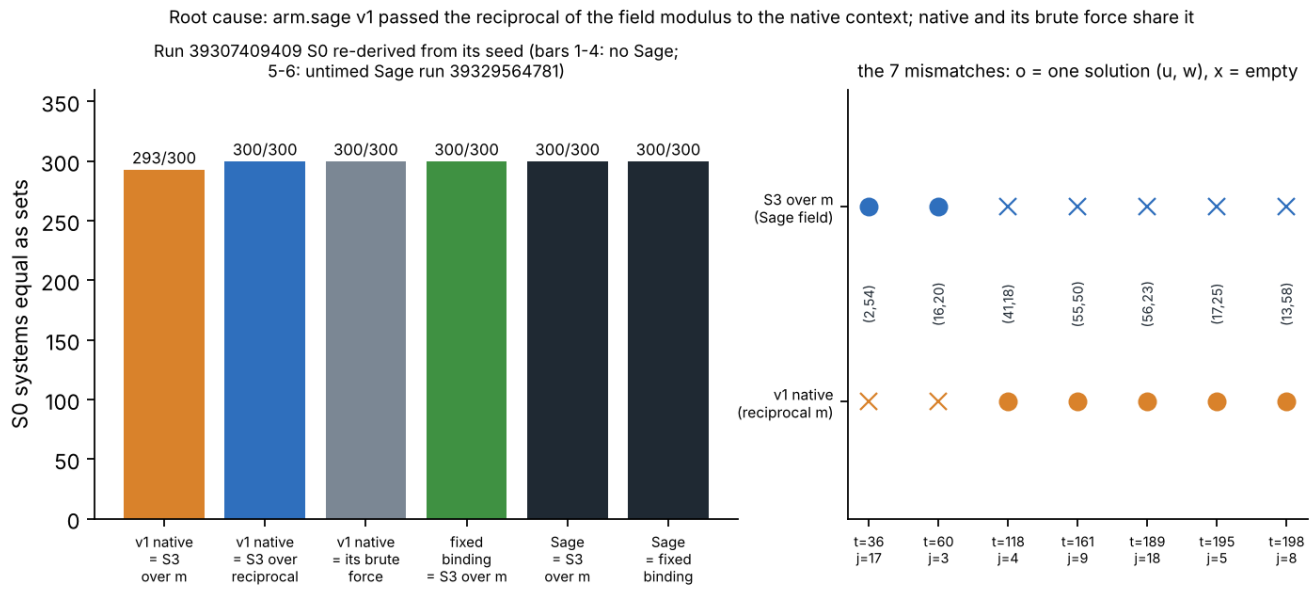


Figure 1: Run 39307409409's S0 re-derived from its seed, with the untimed Sage check. Source: `../make_figures.py s0_diagnosis`; vector: `../figures/s0_diagnosis.svg`.

Which side is wrong, and why

- **The cause.** `run.sage` defines `MODULUS = list(reversed([1, 1, 0, 1, ..., 0, 1, 1]))`, which lists the coefficients lowest degree first. That is the order in which `PolynomialRing(GF(2))(MODULUS)` reads them, so Sage's field is $F_2[z]/(m)$ with $m = z^{19} + z^{18} + z^{16} + z^{11} + z^8 + z^6 + z^4 + z + 1$ (integer 854355). v1 `arm.sage`'s `Native.__init__` read the same list highest degree first (`mod |= 1 << (len - 1 - i)`). It therefore passed the reciprocal $z^{19} + z^{18} + z^{15} + z^{13} + z^{11} + z^8 + z^3 + z + 1$ (829707) to `tau6_new`. The native context worked in a different field.
- **Why native agreed with its own brute force.** `tau6_solve_bruteforce` takes the modulus from the same context. The two share no solving code, but they share the wrong field, so the S0 check "native == brute force" could not see the error. The question "does the brute force share the bug?" has the answer: it shares the input, not the code.
- **Why only 7 of 300 differ.** A random system has $4{,}096 / 2^{19} \approx 0.008$ expected solutions, so almost every system is empty in both fields. Only systems that are non-empty in one field and empty in the other can differ.
- **Evidence, pure Python and the archived library (no Sage).**
  - The re-derivation reproduces every count of the archived S0 row: 293 set-equal, 300 brute-force-equal, 2 non-empty (Sage's count; the direct evaluator over m also finds exactly 2), 0 multi.
  - The v1 native output equals the direct S3 evaluator over the **reciprocal** polynomial on 300 of 300 systems.
  - With the modulus passed correctly, the same archived library equals the direct evaluator over m on 300 of 300, and so does its brute force.
- **Why the local checks missed it.** v1's local checks (`smoke/`, no Sage) used their own binding in `local_replay.py`, which passes `rho_frob.MODULUS_INT`, the correct integer. The binding in `arm.sage` can only be loaded by Sage, so no local check ever ran it. The archived replay (10 of 10 equal to run 38520305068) and the panel check were therefore correct, but they tested a different binding.
- **The Sage side is correct, confirmed directly in Sage.** The untimed diagnostic run 39329564781 ran `sage_diag.sage` in the baseline's image (SageMath 10.10, pinned digest) at `2e8d3ea9`, and recorded every system (`sage_run_39329564781/`). On the 300 v1 S0 systems:
  - Sage's variety equals the direct S3 evaluator in Sage's field on 300 of 300.
  - It equals the native variety with the modulus passed correctly on 300 of 300.
  - It equals the v1 binding's output on 293 of 300, and the 7 differences are exactly the table above.

  Sage's `variety()` over the Boolean ring therefore returns every F_2-point and nothing else. None of the seven involves $u = 0$ or $w = 0$, and `TauField`'s conversion agrees with the direct evaluator on every system. No Sage-side explanation is needed, and the data supports none.

- **The exactness argument.** The claim "linear in w for fixed u" holds: the fixed binding equals the direct evaluator on all 300 v1 systems, on the 307 v2 S0 systems, and the Rust tests check it on 400 more. It also holds in the reciprocal field, which is why v1's native output was a correct variety of the wrong field.
- **The S0 gate.** It compared the right thing: Sage's and native varieties as sets. Its only weakness was that its second reference shared the binding's field. v2 adds a reference that uses Sage's field arithmetic and a field check in every native process.

A second finding: Sage's order of a multi-solution variety

The same Sage run recorded Sage's order on all 2,375 systems of the field with two or more solutions (every (xR, j); `multi_systems.json`, `multi_list.py`).

- **Sage's order.** `variety()` returned all 2,375 ascending in (w, u), that is, by $w_5 \dots w_0\ u_5 \dots u_0$ as a binary number.
- **v1's order.** v1's native order, ascending (u, w), agrees with Sage on only 176 of the 2,375.
- **Why v1's argument held only on v1's panel.** v1 argued that the order was harmless because its panel met only shift-0 swaps. The v2 panel meets three non-swap systems: (xR, j) = (187574, 3), (264110, 18) and (306698, 16). Both solutions of each are in the V′ base (`../smoke/local_v2.json`). Sage's order differs from ascending (u, w) on the first two, so B and N would have returned different relations there and failed the per-point relation gates.
- **v2 returns Sage's order.** The native variety is now sorted by the written word u | w « 6, which is ascending (w, u). This matches Sage on 2,375 of 2,375 multi-solution systems and on all 607 S0 systems (`order_check.py`, `order_check.json`). `cargo test` pins 15 recorded Sage orders.

Is the archived n = 19 baseline affected?

No. The baseline (run 38520305068, `tau.sage`) uses Sage's field and Sage's Groebner solve throughout and never loads the native code. The v1 local replay, with a correctly passed modulus, reproduced its tau6 targets, `gb_calls`, `spurious`, relations and logs on 10 of 10 scalars. So did the v2 replay with the v2 binding (`../smoke/local_v2.json`). Those counts come from Sage's solve, so Sage's solve agrees with the correct native variety on every system of that run.

Fix (protocol v2)

- `../native_binding.py` is a plain-Python binding imported by both `arm.sage` and the local checks. It takes the modulus as an integer built lowest degree first.
- **Field check in every native process.** `tau6_modulus` must equal Sage's `F.modulus()`, and 256 products through `tau6_mul` must equal Sage's. The check runs after the row's timed intervals. With the v1 modulus it fails on 256 of 256 products, which is the negative control in `s0_v2_local.json`.
- **S0 v2.** It runs the 7 systems above plus 300 new ones, compares four evaluators (Sage Groebner, native, native brute force, and direct S3 in Sage's field), and records every system.
- **Rust tests.** `cargo test` now has 7 tests. They cover the 7 systems over m, the v1 failure reproduced over the reciprocal, 15 recorded Sage orders, and the field product checked against a reference and through the C ABI. The enumeration is unchanged; only the order in which solutions are written changed, to Sage's.
- **S0 v2 on the runner.** S0 also checks Sage's order against the native order on all 2,375 multi-solution systems.

Locally, without Sage (`s0_v2_local.py`): on the 307 v2 S0 systems, the fixed binding equals the direct evaluator and the brute force on 307 of 307. The field check passes, and the v1 binding is rejected.